Cache-Aware Request Routing for Distributed RAG-Based LLM Serving: Design, Calibration, and Empirical Findings

Saadet Cansu Baktıroğlu
Dept. of Artificial Intelligence Engineering
TOBB University of Economics and Technology
Student ID: 231401023
sbaktiroglu@etu.edu.tr

Umut Baran Boztaş
Dept. of Computer Engineering
TOBB University of Economics and Technology
Student ID: 231101064
uboztas@etu.edu.tr

Demir Doruk Dilek
Dept. of Computer Engineering
TOBB University of Economics and Technology
Student ID: 231104088
demirdoruk.dilek@etu.edu.tr

***Abstract*—Retrieval-augmented generation (RAG) workloads served across a pool of large language model (LLM) replicas create a routing problem that plain load balancing ignores: which replica already holds the retrieved context in its KV prefix cache. We present a cache-aware request router placed transparently in front of an OpenAI-API-compatible vLLM deployment. Seven selectable strategies share one framework, ranging from cache-blind baselines to worker-specific joint reordering, two experimental overlap-adaptive modes, and a semantic candidate pre-filter. Calibration experiments show that deployment-dependent constants must be measured on the target hardware; the ordering and timing sweeps' fixed, uncalibrated tracker capacity is a limitation. In a replicated four-arm ordering ablation at retrieval depth ten, cache-informed greedy ordering raises the mean per-request cached-token fraction from 65.0% to 73.1% and reduces median time to first token (TTFT) by 28% relative to canonical ordering; its primary answer-quality score is not separated from relevance order under the prespecified spread rule. Tracker-capacity calibration can reverse a routing ablation's conclusion, while replicated timing tests show that completion-time cache bookkeeping deteriorates as concurrency rises whereas dispatch-time bookkeeping remains stable. Trace-driven calibration also corrects the drift reference and CUSUM threshold, although the resulting detector remains only moderately discriminative and does not control routing. Live two-worker tests cover the flat-prompt policies and the base two-phase worker-and-order path. That path establishes protocol operation, not a performance advantage: `per_worker_tree` is absent from the four-arm ablation, whose greedy arm uses one global, worker-blind tree. Whether worker-specific trees outperform that global baseline therefore remains open. Semantic filtering and token-weighted worker scoring remain prototypes, and current two-phase TTFT excludes the preliminary decision call. These boundaries are reported explicitly rather than treated as completed evaluations.**

***Index Terms*—LLM serving, KV cache, prefix caching, request routing, retrieval-augmented generation, load balancing, vLLM**

I. Introduction

Serving large language models behind a pool of replicas is now standard practice for throughput and availability. When the workload is retrieval-augmented – each request's prompt is built from a fixed system preamble followed by a handful of retrieved document chunks – the serving engine's prefix KV cache can eliminate most of the prefill cost for a request *if* it is routed to a replica that already holds a matching prefix in cache. A router that is blind to this fact (round robin, or load balancing on queue depth alone) discards a large, freely available source of latency reduction: which replica currently holds which prefix is entirely a function of routing history, so the router itself controls the very signal it would need to route well.

This paper describes a cache-aware router built as a transparent reverse proxy in front of N vLLM [2] replicas, together with a progression of routing strategies of increasing sophistication. The starting point is a single scoring function,

$$\text{score}(w) = \alpha \cdot \text{cache\_gain}(w) - \beta \cdot \text{load}(w),$$

of which round-robin, pure load balancing, and a cache-aware policy with an adaptive guard band are all degenerate special cases ($\alpha{=}\beta{=}0$ plus rotation; $\alpha{=}0$; and the full form, respectively). We extend this baseline in four directions, reporting live and isolated validation separately:

1) **Worker-specific joint reordering.** A per-worker variant of the CacheWeaver [3] knowledge tree replaces DualMap's [4] dual-hash candidate selection, which we show degenerates to a no-op at $n{=}2$ replicas – exactly the deployment size used throughout this work. The live experiment validates the joint protocol, while its performance relative to one global tree remains an explicit open comparison.
2) **Overlap-adaptive weighting.** A threshold switch and an EWMA-driven weighting mode are implemented, with CUSUM used as a drift diagnostic. The detector is calibrated offline on arrival-ordered workload traces, but its state and the per-worker mode's input stream still differ; no controlled live adaptive ablation is claimed.
3) **Semantic candidate pre-filtering.** A pre-retrieval embedding classifier prototype can rank replicas before the

$O(k^2)$ reorder step. Query text is not yet carried by the live two-phase endpoint, and at the present $N{=}2$, top-$k{=}2$ setting it cannot reduce the candidate set.

4) **Calibration and controlled ablations.** Load reference, average prompt size, SLO/rebalancing thresholds, drift target, CUSUM threshold, and tracker capacity are measured rather than borrowed. Repeated ordering and bookkeeping sweeps rotate arm order, cold-reset both engines and the router, and verify that the replay client keeps its requested arrival schedule.

The calibration discipline turns out not to be a formality. Section VI reports a load point where recalibrating `TRACKER_CAPACITY` reverses which of two policies wins, and a one-line tie-handling defect that always preferred the hash-assigned first candidate. The latter is a plausible contributor to a stable asymmetric live split, but the current evidence does not establish it as the sole cause.

The ordering ablation further separates cache-informed ordering from ordering rules that do not observe cache state. At retrieval depth ten, the retriever's relevance order outperforms both canonical sorting and a session-history prefix, while the cache-informed greedy policy is the only arm that improves on relevance order. This reframes the empirical question from whether chunks should be reordered to whether the ordering rule has an accurate cache signal.

Critically, the four-arm ordering ablation evaluates one global, worker-blind greedy tree under a fixed router; it does not include the paper's `per_worker_tree` strategy. It therefore shows that cache-history-informed ordering can help, but not that worker-specific joint routing and ordering adds value beyond global greedy.

The question at the centre of this work can be stated directly: in a distributed LLM serving cluster, can reordering a RAG request's retrieved document set for cache-friendliness be optimised *jointly* with the worker-selection decision, so that cache-affinity and load-balancing are pursued together rather than traded off – and, critically, *under which traffic conditions* (which retrieved-document-overlap regime) does that joint optimisation actually pay off, and under which does it disappear? The second half of that question matters as much as the first: this paper deliberately asks *under what conditions* the approach wins rather than claiming it wins unconditionally, in keeping with CacheWeaver's [3] own finding that reordering gains depend strongly on document-overlap rate and vanish in low-overlap regimes such as HotpotQA.

The present evaluation answers only a prerequisite question: whether a history-informed order can outperform cache-oblivious orders under fixed routing. It does not yet answer the stronger joint question of whether per-worker trees outperform one global tree; that remains a system-design hypothesis and the first item of future work.

II. Related Work

Three largely separate literature threads intersect with this work: distributed cache-affinity routing, RAG-specific cache reuse, and adaptive/semantic parameter tuning. Table I positions the systems most relevant to the implementation; vLLM [2] underlies the serving engine and is discussed separately at the end of this section.

A. Distributed Cache-Affinity Routing

Preble [6] routes requests sharing a prefix to the same replica to maximise KV-cache reuse, maintaining a precise prefix tree per replica and switching from prompt-aware to load-aware scheduling once a request's prefix-hit rate falls below a threshold. Mooncake [7] disaggregates prefill and decode into separate clusters around a KVCache-centric scheduler that balances throughput against latency SLOs. NVIDIA Dynamo [8] implements a production KV-cache-aware router that scores replicas on a combined decode-cost/prefill-cost estimate derived from cache overlap. DualMap [4] identifies a shared weakness across this line of work – oscillating between a single cache-affinity objective and a single load-balancing objective in one shared mapping space – and proposes a "power of two choices"-style fix: each request is mapped to two independent candidate replicas via two independent hashes, with SLO-aware routing and hotspot-aware rebalancing arbitrating between them.

This thread solves worker selection but is *not RAG-specific*: it treats the prefix as a fixed session history and does not address the fact that retrieval returns a *set* of documents whose order can differ from one query to the next even when the set itself overlaps heavily with a previous query's.

B. RAG-Specific Cache Reuse

RAGCache [9] builds a knowledge tree over retrieved document sequences and caches intermediate states across the GPU/host memory hierarchy, additionally overlapping retrieval with speculative generation. CacheBlend [10] and TurboRAG [11] target the case where reusable chunks do *not* form a contiguous prefix at all, fusing precomputed KV caches (selectively recomputing only a small subset of tokens to preserve generation quality) rather than relying on prefix alignment. Cache-Craft [12] shows that plain prefix caching degrades substantially on production RAG traffic and proposes identifying which chunk-caches remain valid, cheaply repairing the ones that do not. CacheWeaver [3] – the mechanism this paper's `per_worker_tree` strategy builds on – takes a different, engine-unmodified approach: a lightweight prompt-layer method that reorders retrieved chunks into a cache-friendly sequence via a greedy prefix-tree walk (its Algorithm 1), reported to reach 97.5% of an oracle ordering's reusable-prefix depth.

This thread addresses RAG-specific cache reuse; CacheWeaver is the member that specifically reorders retrieved evidence. The listed systems assume a single serving instance and do not include distributed worker selection.

C. Adaptive Tuning and Semantic Approaches

GORGO [13] models per-request cost as the sum of network, queueing, and prefill terms and tunes routing weights

online via an evolutionary strategy against a held-out tuning window; it is network-topology-aware but not RAG-specific and requires black-box search rather than a closed-form update. The semantic-caching family – GPTCache [14], SCALM [15], and MeanCache [16] – compares query embeddings and, on a sufficiently similar match, returns a cached *answer* directly, skipping computation entirely; this operates one layer above KV-cache reuse or worker routing and carries a distinct risk profile (a false-positive match returns a wrong answer outright, rather than merely costing extra prefill).

D. Positioning of This Work

TABLE I
RELATED WORK COMPARED ALONG THREE AXES

System	Dist. routing	RAG cache reuse	Adaptive
Preble [6]	✓	×	partial (threshold)
Mooncake [7]	✓	×	×
Dynamo [8]	✓	×	×
DualMap [4]	✓	×	× (fixed SLO)
RAGCache [9]	×	✓	×
CacheBlend [10]	×	✓	×
TurboRAG [11]	×	✓	×
Cache-Craft [12]	×	✓	×
CacheWeaver [3]	×	✓	×
GORGO [13]	✓	×	✓(evolutionary)
Semantic caching	×	×	×
This work	✓	✓	✓(overlap-adaptive)

This paper combines distributed routing (Section III-A/B/D) with RAG-specific reordering (Section III-C) *in one system*: retrieval output is first reordered for cache friendliness, and that ordering is then evaluated per-replica and optimised jointly with worker selection, rather than as two independent stages. To our knowledge no system in the surveyed literature addresses this specific dependency – whether to reorder before or after choosing a worker changes which ordering is actually optimal, since two replicas' caches generally disagree on what is cache-friendly (Section III-C). Techniques from Section II-C above (adaptive tuning and semantic selection) are borrowed as extended evaluation components layered on top of this core design, not presented as this paper's primary contribution.

RAGRoute [5] is related in spirit but orthogonal in target: it uses a lightweight classifier before retrieval to choose which federated data sources to query. It does not decide which LLM replica's KV cache is most relevant to an incoming query. Section III-F describes the prototype semantic pre-filter explored for that distinct decision.

Serving infrastructure. vLLM [2] provides the underlying PagedAttention KV-cache engine and the OpenAI-compatible API surface this router proxies. All prefix-cache accounting in this work is calibrated against vLLM's own reported counters (`prompt_tokens_details.cached_tokens` and the Prometheus `vllm:prefix_cache_*` series) rather than assumed.

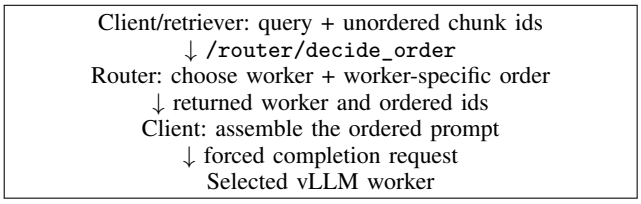

Fig. 1. Current client-mediated two-phase path. The client applies the returned order; the completion-phase timer starts after the decision call in the present replay harness.

III. SYSTEM DESIGN

A. Architecture

The router is a FastAPI reverse proxy exposing the same `/v1/chat/completions`, `/v1/completions`, and `/v1/models` surface as vLLM itself, so that a client (in our deployment, an Open WebUI instance) cannot distinguish it from talking to a bare engine – streaming responses are forwarded byte-for-byte rather than buffered. A background poller scrapes each worker's `/metrics` endpoint on a fixed interval and maintains a per-worker `WorkerState` snapshot (queue depth, running-request count, KV-cache utilisation, prefix-cache hit counters); routing decisions read this snapshot rather than querying workers synchronously on the request path.

Because vLLM exposes aggregate cache statistics but not "do you currently hold a cache entry for this specific prefix", the router keeps its own approximation: for each worker, an LRU of block-aligned prefix hashes it has been sent (`PrefixTracker`), hashed in the same fixed token-block granularity vLLM itself uses. Two independent bookkeeping conventions are supported for *when* a block is recorded as cached – at dispatch time (optimistic: the block will be resident once prefill starts) or at completion time (conservative: only once the engine has actually finished) – and Section VI-E reports the result of ablating between them.

B. Strategy Framework

All registered strategies share a common interface, `select(context, snapshot) -> Decision`, so that switching the active policy is a configuration change rather than a code change. Five policies use the ordinary one-phase proxy path:

- **round_robin**: $\alpha{=}\beta{=}0$, plus a rotating tie-break; the floor every other strategy is measured against.
- **least_loaded**: $\alpha{=}0$; pure, cache-blind load balancing on a saturating load-normalisation $\text{load\_norm}(w) = \text{load}(w)/(\text{load}(w)+\text{LOAD\_REF})$, chosen over a clipped $\min(\cdot, 1)$ form because clipping destroys the policy's discriminative power once both replicas exceed the reference load (Section IV-C).
- **cache_aware**: the full scoring function plus an *adaptive guard band*, $\delta = \delta_0 \cdot (1 - \overline{\text{kv\_usage}})$: the winning replica by score is overridden back to the least-loaded replica if its load exceeds the minimum by more than δ. This prevents the classic failure mode of pure cache-affinity

routing, where the replica holding a popular prefix keeps attracting traffic until its queue makes it the worst choice by latency despite remaining the best choice by cache.
- **adaptive_cache_aware**: `cache_aware` with β and δ_0 driven by an overlap EWMA, while a CUSUM detector records diagnostic drift alarms (Section III-E).
- **cacheweaver_dualmap**: the DualMap-inspired two-choice prototype described in Section III-D.

The two remaining registry entries, `per_worker_tree` and `semantic_per_worker_tree`, expose the separate decision capability described next. This registry taxonomy must not be confused with the arms of any one experiment: Section VI-J isolates order under fixed routing, while Section IX defines the missing system comparison.

C. Worker-Specific Joint Reordering (`per_worker_tree`)

DualMap's replica candidate-selection step (its Eq. 5) hashes the reordered chunk sequence through two independent consistent-hashing rings to obtain a candidate pair (r_1, r_2), with a collision rule $r_2 \leftarrow (r_1 + 1) \bmod n$ when the two rings agree. At n=2 replicas – the deployment size used throughout this work – this collision rule is not an edge case: it is provably always triggered or already trivially satisfied, so $\{r_1, r_2\} = \{0, 1\}$ on every request. The dual-hash candidate-selection machinery is therefore a mathematically guaranteed no-op at this scale, and any cache-affinity signal has to come from elsewhere in the pipeline.

`per_worker_tree` replaces the hash-and-select step entirely. Each worker maintains its own independent CacheWeaver knowledge tree; for every candidate worker, `best_reorder_for()` computes both that worker's own best reordering of the retrieved chunks *and* a normalised cache-gain figure against that specific tree, and the scoring function from Section III-B picks the winner directly – there is no candidate-pruning step to degenerate. This is a genuine change in what "the right chunk order" means: it is no longer a single, worker-independent quantity computed once and then assigned to whichever replica is chosen, but something that emerges jointly with the worker decision, since the two replicas' trees generally disagree about which order is cache-friendly.

Because the winner and its optimal chunk order are decided together, a client-mediated two-step protocol is used (Fig. 1). The client first calls `/router/decide_order` with the unordered, retrieval-order chunk ids and receives back {worker, ordered_chunk_ids}; it then builds the prompt in that returned order and issues the completion with an `x-router-force-worker` header. The router forwards this already ordered prompt to the named worker. Phase-one bookkeeping is optimistic: the tree is updated before phase two succeeds, so the two HTTP calls are not transactional.

1) Token-weighted cache gain: The router module has an optional token-weighted path, gain = hit_tokens/total_tokens, for cases where the caller supplies a chunk-id-to-text mapping. It reuses the existing tokenizer and a synthetic regression test confirms that few large chunks and many small chunks can select different workers. This path is *not live-integrated*: `/router/decide_order` currently accepts only chunk ids, and the strategy calls the module without chunk texts. Consequently each chunk has weight one on the live HTTP path and all reported live `per_worker_tree` traffic remains chunk-count-based. Section VI-G is therefore an isolated functional test, not a measured live improvement.

2) Quality protection (`protect_top_k`): Pure greedy reordering can push a highly relevant but uncached chunk to the end of the prompt while pulling a barely relevant but cached chunk to the front – a risk CacheWeaver's own evaluation does not fully rule out (its Table 7 states only that reordering *did not measurably hurt* answer quality in their setting, not that it never can). `protect_top_k` pins the first K retrieval-order chunks in the returned list and reorders only what follows; K=0 reproduces the original behaviour. A regression test verifies this positional property. It does not establish that the post-pinned traversal remains one cache-contiguous prefix when a pinned chunk is absent from the tree, nor does it establish answer quality; both require end-to-end validation.

D. The `cacheweaver_dualmap` Baseline and a Tie-Break Defect

DualMap's official implementation (https://github.com/ASISys/DualMap, MIT-licensed, which also incorporates Preble) is publicly available, but was not used directly: it is written against an 8+-GPU, 512 GB-DRAM reference deployment and consumes Mooncake-format traces (fixed prefixes, no real retrieval step), neither of which matches this project's two-machine RAG deployment with genuine per-query retrieval. The local `cacheweaver_dualmap` strategy is therefore a *DualMap-inspired two-choice prototype*, not an equivalent reimplementation. It uses two independent BLAKE2b-modulo mappings rather than consistent-hash rings, estimates recompute cost with a fixed 200-token-per-chunk placeholder, and derives pending-token load from vLLM's waiting-request gauge. On the documented vLLM V1 setup that gauge can remain zero under load, so SLO/rebalancing activation has not been demonstrated. Moreover, although the inner prototype computes a CacheWeaver order, the enclosing strategy does not propagate that order to the proxy's prompt-rewrite field; vLLM therefore receives the original prompt order in the reported one-phase runs. Results bearing this strategy name evaluate its current two-choice routing and internal bookkeeping, not live CacheWeaver reordering or a faithful DualMap run.

An earlier internal version also used one shared tree for both replicas, making their recompute estimates identical. Per-replica trees fixed that error but exposed a separate tie defect: non-strict comparisons always selected the hash-assigned r_1 when estimates tied. The current implementation randomises this candidate tie; Section VI-F describes the corresponding GPU-free regression test.

E. Overlap-Adaptive Weighting

A common formula underlies the adaptive modes: Jaccard overlap $J(A, B) = |A \cap B|/|A \cup B|$. The cur-

rent call sites do *not*, however, observe the same stream. `adaptive_cache_aware` compares adjacent requests within each session, whereas `per_worker_tree`'s modes compare globally adjacent requests and initialise the first comparison against an empty set. The offline tool reports both notions separately (Section IV-D); sharing a formula does not make their calibrated values interchangeable.

Threshold mode. A one-line switch: α=0.2, β=0.8 when J falls below a configured threshold (locality is a weak signal, weight load instead), and α=0.8, β=0.2 otherwise, with $\alpha + \beta$=1 by construction.

EWMA with CUSUM diagnostics. An exponentially weighted moving average tracks a smoothed overlap estimate $D_t = \lambda J_t + (1 - \lambda) D_{t-1}$. This EWMA, together with the fixed configured D_{target}, feeds $\beta_{\text{live}} = \beta_{\text{base}} \cdot \text{clip}\left(1 + k \cdot \frac{D_{\text{target}} - D_t}{D_{\text{target}}}, [0.3, 3.0]\right)$, while α is held fixed. A CUSUM detector is updated alongside it and recalibrates its own alarm reference, but the returned alarm and recalibrated reference are not consumed by routing. CUSUM is therefore observational telemetry in the current code, not a second decision input. Offline calibration now reconstructs the exact arrival-ordered, within-session stream observed by `adaptive_cache_aware`; at retrieval depth ten this gives `D_TARGET=0.322` and favours `CUSUM_H=0.30` over the previous 0.20 default. The per-worker mode still consumes global adjacency, so the calibrated target is not transferable to that path. GPU-free tests verify that `off` reproduces the fixed baseline and that threshold/EWMA inputs can change a synthetic decision; no live comparative result is claimed.

F. Semantic Candidate Pre-Filtering

For deployments with more than a handful of replicas, `per_worker_tree`'s $O(k^2)$ per-candidate reorder becomes $n \cdot O(k^2)$ across all healthy workers – cheap at n=2, not necessarily cheap beyond it. `SemanticWorkerRouter` addresses this independently of locality-based weighting: each worker's recent query traffic is summarised as an EWMA centroid in embedding space (`multilingual-e5-base`, the same checkpoint used for corpus retrieval, loaded once per router process); an incoming query is embedded and ranked by cosine similarity against each worker's centroid, and only the top-k ranked workers are passed to `per_worker_tree`'s comparison. A worker with no observed traffic yet is assigned the *average* similarity of workers that do have a centroid – neither a fixed low score (which would permanently exclude new or recently restarted workers) nor a fixed high score (which would make every cold worker an automatic favourite) – and the candidate set falls back to every healthy worker whenever the semantic favourites happen to be unhealthy, so the mechanism cannot starve a request. Loading the embedding model is forced synchronously at router start-up rather than being paid by whichever live request happens to arrive first.

The current deployment cannot exercise the intended prefilter. The two-phase endpoint carries chunk ids but not query text, so that path falls back to every healthy worker. In addition, the default `SEMANTIC_TOP_K=2` equals the entire two-worker pool and cannot prune candidates even if query text is supplied. The ranking logic is tested in isolation and model warm-up was observed separately, but a semantic filtering benefit requires a revised protocol and $N > k$.

IV. Implementation Details

A. Corpus and Trace Construction

The RAG corpus is built from the publicly available TQuAD dataset [1], a SQuAD-format Turkish question-answering resource (681 articles, 8,308 questions). Each question already points at exactly one source paragraph, so using the paragraph as the retrieval chunk gives retrieval ground truth "for free". Paragraphs are split at sentence boundaries only when they exceed a maximum chunk length, and every chunk is padded (in token space, not by appending raw text – tokenizers merge whitespace unpredictably) to a multiple of vLLM's KV-cache block size, since a chunk boundary that does not align with a block boundary produces a different block hash even when the underlying content is shared, silently breaking the prefix chain the whole system depends on. The resulting corpus contains 2,619 chunks. Before padding, lengths range from 9 to 754 tokens (mean 182.2, σ=108.5); after padding they range from 16 to 768 tokens (mean 189.6, σ=108.4). The variation motivates the token-weighted prototype of Section III-C.1, but that prototype is not yet connected to live requests.

Synthetic request traces are generated once per experiment and replayed identically across every strategy under comparison, so that strategies are never compared against different workloads. Two independent knobs control cache locality: popularity skew across titles (a Zipf distribution with configurable exponent s) decides *which* chunks are asked about repeatedly, and session structure (several questions drawn from one title, arriving with realistic think-time gaps) decides *when* repeats occur relative to each other – a popular chunk asked about ten times over an hour is not locality if the cache empties between hits. An optional popularity-drift knob rotates the ranking over the course of a trace, exercising the scenario an adaptive policy is meant to handle and a static one is meant to degrade gracefully under.

B. Software and Deployment Platform

The implementation is written in Python and uses FastAPI and Uvicorn for the OpenAI-compatible reverse proxy, `httpx` for asynchronous stream forwarding and worker polling, and `prometheus-client` for router instrumentation. Each upstream runs vLLM with Qwen2.5-7B-Instruct; the two workers communicate with the router over a private Tailscale network. Hugging Face Transformers supplies the serving-model tokenizer, `multilingual-e5-base` supplies corpus and query embeddings, and NumPy stores and scores the offline embedding matrix. The submission also contains replay, trace-generation, calibration, quality-scoring, schedule-lag, and GPU-free regression scripts. Runtime parameters, worker addresses, strategy selection, and calibration constants

are exposed as environment variables rather than requiring source edits.

C. Calibration Discipline

TABLE II
CALIBRATED SETTINGS AND DOCUMENTED EXPERIMENTAL EXCEPTIONS

Constant	Value	Measured via
`LOAD_REF`	16 concurrent requests	Kleinrock power (throughput/latency) peak, single-worker concurrency sweep, Qwen2.5-7B, ∼1700-token prompts
Mean prompt size	2,392 tokens	Mean prompt token count, n=800 live RAG requests, top-k=10
DualMap-inspired SLO / rebalance thresholds	38,272 / 57,408 tokens	`LOAD_REF` × mean prompt tokens; 1.5× ratio preserved from DualMap's relationship
`D_TARGET` (drift reference)	0.322	Arrival-ordered within-session Jaccard stream, top-k=10, 2,170 stable observations
`CUSUM_H`	0.30	Grid calibration on nominal and drift-0.1 traces; 26.73 nominal alarms/1,000 and 2.20× separation
`TRACKER_CAPACITY`	5,840 blocks calibrated; 50,000 in ordering/timing sweeps	Smaller worker's KV-cache capacity; sweep exception held constant across arms

Table II separates calibrated values from a deliberate experimental exception. Semantic top-k, centroid learning rate, CUSUM margin K, EWMA rate, and `protect_top_k` retain explicit prototype defaults. The source default for `TRACKER_CAPACITY` is 50,000; calibrated routing experiments require `ROUTER_TRACKER_CAPACITY=5840`. The new ordering and timing sweeps were instead run at 50,000 in every arm. This does not vary within either comparison, but it can affect absolute cache-hit and tracker-accuracy values and is therefore not hidden as a calibrated setting. Section VI-B shows why the distinction matters.

D. Overlap Characterisation Tool

A standalone tool computes the Jaccard overlap between consecutive requests' retrieved chunk sets directly from a corpus and trace, without touching the router or a live engine (retrieval alone is enough). It reports two distinct notions of "consecutive" separately, because they answer different questions: *session-adjacent* overlap (turn k to turn $k{+}1$ within one session) reflects what a prefix cache actually sees if the router keeps a session pinned to one worker, while *global-adjacent* overlap (arrival-order adjacency, ignoring session boundaries) reflects the raw opportunity the router faces before any session-affinity policy is applied. This distinction turns out to matter a great deal (Section VI-A).

V. EXPERIMENTAL SETUP

A. Deployment and Protocol

All live experiments use a two-worker vLLM deployment (Qwen2.5-7B-Instruct) connected over a private network, driven by a replay harness that fires each trace request at its scheduled offset regardless of whether earlier requests have completed – waiting for completions would collapse queue depth to one and prevent the load term of the scoring function from ever activating. Every run validates the router's own belief about cache state against the engine's ground truth, read from `usage.prompt_tokens_details.cached_tokens` in the streamed response, rather than assuming the two agree.

For the two-step path, the current replay harness records `sent_s` before `/router/decide_order` but starts its TTFT and total-duration clock only immediately before the completion request. The reported two-step TTFT therefore excludes decision, reordering, and the first HTTP round trip. It validates completion-path behaviour but is not an end-to-end latency directly comparable with a one-phase strategy.

The replicated ordering and bookkeeping ablations use `trace_hot.jsonl` (Zipf s=1.5, session length four), 300 requests, top-k=10, the Hugging Face tokenizer, and a fixed `cache_aware` routing strategy. Before every arm, both vLLM engines are restarted to clear their persistent prefix caches and the router is restarted to clear `PrefixTracker`. Arms are interleaved repeat-major and their order is rotated between three repetitions, limiting slow hardware drift from favouring one policy. The replay schedule is verified independently: across all 12 timing runs, schedule-lag p99 is at most 6 ms, below the 1 s rejection threshold. Ordering changes only chunk order and retains one request round trip; the router, join delimiter, prompt template, and request schedule remain fixed.

B. Evaluation Metrics

The planned system comparison and its load/locality extensions (Section IX, items 1–2) are designed to report five metrics:

- **TTFT (Time to First Token).** For one-phase runs, time from request dispatch to the first generated token, reported at p50 and p95. The current two-step measurement is completion-phase TTFT; an end-to-end timer remains to be added before cross-path comparison.
- **Throughput / goodput.** The current sweep scorer computes raw observed throughput. SLO goodput (e.g. requests with TTFT <5 s per second) is the intended metric but is not implemented yet.
- **Cache reuse (token-weighted).** The fraction of prompt tokens served from cache rather than recomputed, taken from vLLM's response accounting. This engine metric is token-based even though live `per_worker_tree` worker scoring is still chunk-based. The current ordering scorer macro-averages this fraction across requests; it is therefore not identical to aggregating cached and prompt token counts over the full run.

- **Load balance (CV).** The coefficient of variation of request counts across workers; lower means a more even split.
- **Answer quality.** Because TQuAD answers are short spans while the model commonly returns a sentence, the primary metric is whether the normalised model output contains the gold span. Token recall, token-F1, and exact match are retained as secondary metrics. The replicated ordering sweep reports mean and sample standard deviation; differences smaller than twice the largest within-arm standard deviation are treated as unresolved.

The dry-run sweep scorer also needs correction before live use: it averages per-request cache fractions rather than aggregating token counts, and currently reports load CV as zero when all traffic reaches one worker because absent workers are omitted. These are implementation gaps, not completed metric results.

VI. Results

Artifact status. The repository includes the overlap scores, trace manifests, experiment drivers, calibration tools, regression tests, and a configuration-recording sweep harness. It does not retain the raw live-run JSONL outputs underlying the numerical live tables. Those values are preserved in the project log and final report but cannot be recomputed from the submitted tree without rerunning the two vLLM workers.

A. Overlap Characterisation

On a 3,000-request trace (830 sessions, baseline popularity skew s=1.0), session-adjacent overlap averaged 0.476 (median 0.500, 90th percentile 1.000; 76.2% of consecutive same-session pairs shared at least one chunk), while global-adjacent overlap averaged only 0.055 (median 0.000; only 7.2% of arrival-order-adjacent pairs shared anything). The gap is the central argument for session-affinity routing: without it, prefix-cache locality is nearly unusable even though it is abundant within sessions.

Sweeping popularity skew from s=0 (uniform) to s=2.0 (heavily skewed) moves global-adjacent overlap 13-fold, from 0.023 to 0.308 – the more concentrated real-world query popularity is (e.g. FAQ-style repeated questions), the more of a chance a cache-blind router has of stumbling onto a hit even without session affinity. Popularity drift has the opposite, and faster-saturating, effect: turning drift on at all (rate 0.1) drops session-adjacent overlap from 0.476 to 0.383 and global-adjacent overlap from 0.055 to 0.030, with almost no further degradation from 0.1 to 0.5. This motivates testing adaptation under drift; it is not itself evidence that either adaptive mode improves live routing.

B. Calibration Reverses an Ablation

Before `TRACKER_CAPACITY` was corrected, the router's belief about how much of each replica's cache it could rely on was computed against an assumption roughly nine times larger than either machine's real capacity. Under that assumption, β=0 appeared to win at 30× load; after recalibrating capacity to the smaller machine's real value (5,840 blocks) and rerunning under an identical protocol (cold restart before every arm), the winner at 30× flips to β=1 (Table III). No pre-calibration value for 35× is retained in the current report, so the demonstrated reversal is specifically the 30× result. It still shows that an uncalibrated capacity constant can invert an ablation's recommendation.

TABLE III
CALIBRATED CACHE-AWARE (β=1) VS. LOAD-BLIND (β=0)

Load	β=1 TTFT p50	β=0 TTFT p50	Winner (calibrated)
30×	0.256 s	0.488 s	β=1 (+47%)
35×	2.404 s	2.107 s	β=0 (+12%)

A secondary, still-open observation from the same experiments: worker split is consistently asymmetric in a strategy-dependent, load-independent way (β=1 runs consistently route more traffic to one physical worker, β=0 runs to the other). `cache_aware`'s own tie-breaking is already randomised, so this is not the same class of defect identified in Section VI-F; it is more likely attributable to a genuine, unmeasured asymmetry between the two physical machines (network latency, GPU clock state) and is left as an open item (Section VII).

C. `cacheweaver_dualmap` vs. `cache_aware`

TABLE IV
HISTORICAL THREE-RUN COMPARISON OF CURRENT STRATEGY LABELS

Strategy	TTFT p50 (3 runs)	Mean	Hit rate
`cache_aware`	0.272 / 0.346 / 0.318 s	0.312 s	66.0–66.6%
`cacheweaver_ dualmap`	0.314 / 0.309 / 0.363 s	0.329 s	66.5–66.7%

A single earlier n=1 run had suggested a 56% TTFT advantage for `cache_aware`; repeating the comparison three times under calibrated thresholds narrows the gap to 5.5%, with overlapping run-to-run variation. A stable worker-split difference also appears: `cacheweaver_dualmap`'s worker split locks to a near-identical ratio across all three runs (∼538/262 each time), while `cache_aware`'s split varies considerably run to run (282/518, 293/507, 349/451). These numbers compare the implemented strategy labels only: Section III-D shows that the former did not apply its computed order to vLLM and may not have exercised its load/SLO branch. Section VI-F identifies a candidate-tie bias as one plausible contributor, not a complete causal explanation.

D. Sensitivity Analysis

Repeating the crossover comparison on a second trace with markedly lower repeat rate (68.8% vs. 91.9% for the original) produces four outcomes in which each policy wins twice (Table V). The clearest pattern is at the highest load tested (35×): the gap between policies grows substantially on both traces, but in *opposite* directions (a 12% win for one policy on one trace and a 54% win for the other policy on the other). A claim of the form "policy X wins at high load" is therefore

TABLE V
β=1 VS. β=0 ACROSS TRACE LOCALITY AND LOAD

Trace	Load	β=1	β=0	Winner
High locality	30×	0.256 s	0.488 s	β=1 (+47%)
High locality	35×	2.404 s	2.107 s	β=0 (+12%)
Low locality	30×	0.322 s	0.240 s	β=0 (+25%)
Low locality	35×	2.018 s	4.370 s	β=1 (+54%)

underspecified without also stating which workload locality regime it was measured under – the benefit of the cache-affinity term is a joint function of load *and* locality, not load alone.

E. Dispatch-Time vs. Completion-Time Bookkeeping

TABLE VI
REPLICATED CACHE-STATE ACCURACY AT TWO REPLAY LOADS (THREE RUNS PER MODE)

Spd.	Mode	Corr.	Rank agree.	Underest.
5×	Dispatch	0.972±0.010	97.1±0.6%	0.0±0.0%
5×	Completion	0.941±0.011	92.5±0.6%	6.2±0.7%
15×	Dispatch	0.980±0.003	98.0±0.0%	0.0±0.0%
15×	Completion	0.806±0.008	79.7±0.9%	29.6±0.8%

Recording a block as cached at the moment a request is *dispatched* is, in principle, optimistic – prefill has not actually happened yet, so the belief could be wrong if the request later fails. Recording at *completion* is, in principle, conservative and therefore expected to be more accurate. The routing-relevant correlation and rank metrics show the opposite and, more importantly, expose a load interaction (Table VI). When replay speedup increases from five to fifteen, dispatch correlation remains near 0.98 with zero observed underestimation, while completion correlation falls from 0.941 to 0.806 and its rank concordance falls from 92.5% to 79.7%. Its underestimate rate simultaneously rises from 6.2% to 29.6%. The correlation gap consequently expands from 0.031 to 0.174. Completion has the smaller absolute bias at speedup five, so the evidence does not support the stronger claim that dispatch is closer on every error metric. The mean biases (tokens) at speedups five and fifteen are respectively $+31.3\pm9.2$ and $+23.7\pm1.0$ for dispatch, versus -5.2 ± 20.3 and -170.3 ± 25.5 for completion.

The proposed mechanism is that completion-time bookkeeping cannot yet see sibling requests dispatched to the same prefix but still in flight when a new request arrives, and their number increases with concurrency. Dispatch-time recording sees them immediately, although it can overestimate if a dispatched prefill later fails. The asymmetric load response and positive dispatch bias in all six new runs are consistent with this explanation but do not isolate it as the only cause. This load-dependent result supersedes the earlier single-run table, whose load condition was not recorded and whose negative dispatch-bias sign was not reproduced under the new protocol. Because these sweeps held the uncalibrated 50,000-block tracker capacity fixed across modes, the comparison supports the mechanism and relative ranking, not an absolute error claim for a calibrated deployment.

F. A Tie-Break Defect and Its Fix

The candidate comparison inside `cacheweaver_dualmap`'s SLO-aware selection step used a non-strict inequality, so that whenever two candidates' estimated recompute cost tied – common, since both per-replica trees are frequently equally cold or equally warm – the hash-assigned first candidate won unconditionally. We replaced both non-strict comparisons in the selection path with an explicit, uniformly random tie-break, matching the convention already used elsewhere in the codebase. A GPU-free regression test constructs 400 equal-cost requests and checks that neither physical worker receives less than 35% of them; unseeded reruns are approximately even. The old code selected the hash-assigned r_1 candidate on every tie, but this does *not* imply a 400/0 physical-worker split because the identity of r_1 varies with the hash key. Whether the fix changes the live split remains unmeasured.

G. Synthetic Token-Weighting Validation

On a synthetic scenario constructed so that one worker's cache holds two large chunks (combined ≈1,300 tokens) and a second worker's cache holds three small chunks (combined ≈9 tokens), count-based cache-gain accounting selects the three-small-chunk worker (3 matched chunks beats 2), while token-weighted accounting selects the two-large-chunk worker (hit_tokens/total_tokens ≈0.99 vs. ≈0.01) – the two accounting schemes disagree on this input by construction, and the token-weighted result better represents potential prefill saving. This validates the optional module path only; the live endpoint does not supply the required chunk texts (Section III-C.1).

H. Embedding-Model Warm-Up Observation

During a 300-request two-worker run (0 failures), the semantic strategy incurred a one-time ∼5–6 second embedding-model load that, before a start-up warm-up call was added, landed on whichever live request happened to arrive first after a router restart. Forcing this load synchronously at start-up (verified by measuring the warm-up call's own duration directly against the duration of the first `predict_candidates` call immediately following it) moves the cost out of the request path. This run did not validate semantic filtering: the two-phase endpoint omitted query text and top-k equalled the two-worker pool.

I. Live Protocol Validation: `per_worker_tree`

The two-phase decide-order protocol was validated against live two-worker traffic for the first time in this round of experiments (an earlier validation used only a single worker, where no real candidate comparison is possible). Across 300 requests: 0 failures, 100% of completions correctly forced to the worker chosen by `/router/decide_order` (`reason=forced_by_decide_order` on every request), and real, non-degenerate traffic splitting (130/170). This validates worker forcing and client-applied ordering across both replicas. The split is not a cache-efficiency or latency comparison

against global greedy. No matched baseline was collected for cache reuse, end-to-end latency, or answer quality. Moreover, the run does not include phase-one latency in TTFT and uses chunk-count rather than token-weighted worker scoring; it is consequently a protocol-correctness result only.

J. Worker-Blind Cache-Informed Ordering Ablation

All four arms below use the same `cache_aware` router and vary only prompt order; none uses `per_worker_tree`. In particular, the greedy arm maintains one global estimated history tree and does not associate a cached prefix with a particular replica.

TABLE VII
ORDERING ABLATION AT TOP-k=10 (MEAN ± SAMPLE STANDARD DEVIATION, THREE RUNS PER ARM)

Order	Mean cached fraction	TTFT p50	Contains gold
Greedy	73.1±0.2%	0.088±0.006 s	79.6±0.4%
Relevance	67.7±0.1%	0.111±0.025 s	79.9±0.4%
Canonical	65.0±0.5%	0.122±0.015 s	77.8±0.2%
`pinned_prefix`	62.9±0.2%	0.161±0.012 s	79.7±0.3%

At top-k=3, a preliminary single run produces only a 2.4 percentage point spread in mean cached fraction (79.1% canonical, 78.7% relevance, 81.1% greedy), so the ordering lever is too narrow to separate the arms. At top-k=10, twelve interleaved runs yield the clear ranking in Table VII. Relative to canonical sorting, the global CacheWeaver-style greedy tree raises the mean per-request cached-token fraction by 8.1 percentage points and reduces median TTFT by 27.9%. Its primary quality score is not separated from relevance order by the prespecified spread rule (79.6% vs. 79.9%). Greedy changes the order on 43.3% of requests, with a mean reported reorder depth of 7.17 out of ten.

The negative `pinned_prefix` result helps identify the relevant mechanism. It places chunks repeated from the previous turn of the same session first, then preserves relevance order for the tail. It engages on 57.0% of requests, matching the 57.15% predicted from trace session structure, yet finishes last on cache reuse. At top-k=10, it pins an average of only 3.38 chunks and never pins all ten. Its contains-gold score is not separated from relevance order under the prespecified spread rule, and its cache result is consistent with the hypothesis that it disrupts cross-session prefix consistency already supplied by the retriever; the experiment does not directly measure that causal pathway. Taken together, these four arms do not show that arbitrary reordering helps: both cache-oblivious imposed orders underperform the untouched relevance order, and only the cache-history-informed greedy policy improves it. This conclusion is limited to the stated trace, retrieval depth, and uncalibrated tracker-capacity setting.

The proposed `per_worker_tree` scheme is absent from Table VII. The four displayed arms isolate prompt order under a fixed `cache_aware` router and a one-phase timer; `per_worker_tree` jointly changes both worker selection and order through a two-phase path. It cannot be treated as an unqualified fifth pure-order arm. Nevertheless, this missing system-level comparison leaves the central incremental hypothesis unanswered: whether per-replica trees improve upon the 73.1% global worker-blind greedy baseline. The 130/170 live run above validates mechanics, not superiority. A controlled comparison requires arm-specific router configuration, identical cold starts and traces, and end-to-end timing of both protocol phases.

K. Trace-Driven CUSUM Calibration

TABLE VIII
CUSUM THRESHOLD CALIBRATION AT TOP-k=10

H	Nominal alarms/1,000	Drift/nominal alarm ratio
0.20 (previous)	108.76	1.52×
0.30 (selected)	26.73	2.20×

The detector was calibrated without GPU inference because its input is a deterministic function of the retrieved chunk sets. The calibration replays the router's actual arrival-ordered, within-session Jaccard stream rather than grouping observations by session: 2,170 stable observations from `trace.jsonl` and 2,139 from an otherwise matched drift-0.1 trace. Retrieval depth changes the stream materially. The measured stable reference is 0.478 at top-k=3 but 0.322 at the top-k=10 setting used by the live experiments, invalidating the previous 0.529 default.

With λ=0.1 and K=0.03, increasing H from 0.20 to 0.30 reduces nominal-trace alarms fourfold while improving separation (Table VIII). Even after calibration, however, 2.20× separation costs alarms on 2.7% of nominal observations; because the nominal trace has no labelled change points, these are treated as a conservative false-alarm proxy rather than ground-truth false positives. Stricter budgets leave too few alarms for a strong claim. Feeding the smoothed EWMA rather than raw Jaccard nearly silences both streams and does not improve discrimination. CUSUM therefore remains diagnostic telemetry, not an empirically validated routing control signal.

VII. DISCUSSION AND LIMITATIONS

The most defensible conclusion is narrower than "one router wins." Calibration and measurement protocol can change the apparent result: the 30× tracker-capacity correction reversed a policy comparison, an n=1 comparison overstated a gap later narrowed by repetition, and the replicated bookkeeping sweep showed a load-dependent failure of completion-time tracking. The ordering ablation similarly supports a conditional conclusion: at top-k=10, only the history-informed global greedy tree improves on relevance order; canonical and session-prefix rules do not. These results favour cache-informed ordering, not reordering in general. The committed overlap data and experiment code are reproducible; the numerical live tables require their missing raw outputs before they can be independently verified.

The implementation audit also separates completed paths from prototypes. Base worker-specific ordering is client-applied and was exercised across both workers, but its timer omits phase one and its scoring is chunk-count-based. Most importantly, it has not been compared against the global greedy arm on cache reuse or end-to-end latency; therefore the paper does not yet establish the incremental performance benefit of worker-specific trees. Semantic pruning, token-weighted scoring, CUSUM-driven policy changes, and a controlled adaptive comparison are not live results. CUSUM's alarm threshold is now trace-calibrated, but alarms still do not affect the routing weights. The DualMap-inspired arm neither applies its computed order to vLLM nor proves that its load/SLO branch activates. These are material scope boundaries, not minor engineering details.

A. Additional Threats to Validity

The remaining threats to validity are:

1) **Missing central performance comparison.** The pure ordering sweep holds `cache_aware` routing fixed, whereas `per_worker_tree` jointly selects a replica and an order. The two-worker run verifies that this protocol executes correctly but does not compare it with the global, worker-blind greedy tree. The paths also use different prompt separators and timing boundaries. A valid system-level comparison requires matched prompt serialisation, a timer starting before `/router/decide_order`, and identical cold-start, capacity, trace, and load conditions. Until then, the central worker-specific performance hypothesis is unresolved.
2) **Two workers and a partial baseline.** At $n{=}2$, both hash candidates cover the whole pool. The local DualMap-inspired strategy also differs from the published system in mapping, migration, cost estimation, and load instrumentation. It must not be interpreted as an evaluation of official DualMap.
3) **Synthetic traffic.** The overlap characterisation (Section VI-A) and every live experiment use a synthetic trace (popularity-skewed, session-structured) generated over the real TQuAD corpus, not captured production traffic. Absolute values (overlap magnitudes and TTFT) may shift under production traffic. The replicated ordering result additionally covers one hot trace, 300 requests, and top-$k{=}10$; top-$k{=}3$ produced only a preliminary null result.
4) **Capacity and aggregation.** The new ordering and timing sweeps held `TRACKER_CAPACITY=50000` fixed, not the calibrated 5,840-block value. Constancy removes a direct arm-level change but does not rule out a capacity-by-policy interaction. Their cache metric is also a macro-average of per-request fractions rather than a run-level cached-token aggregate.
5) **Run reproducibility.** The repository lacks raw live JSONL outputs and a complete hardware/driver manifest. The new sweep scripts record configuration, arm order, and schedule lag, and trace manifests record generation parameters, but numerical tables still require a fresh deployment run. Defaults also differ from some reported settings: tokenizer mode is approximate and tracker capacity is 50,000 unless overridden.
6) **Two-step accounting.** Current TTFT excludes the decision round trip, and phase-one tree insertion can survive a failed phase-two request. Cross-strategy latency and cache-state correctness need an end-to-end, failure-aware protocol.
7) **Prototype calibration.** Semantic top-k and quality-protection parameters remain defaults. The CUSUM calibration uses one synthetic drift strength, and the per-worker adaptive path compares global adjacency against a within-session target; alarms do not affect routing.
8) **Answer quality.** The new quality result validates the global, worker-blind greedy arm under one TQuAD trace and the contains-gold metric. It does not validate worker-specific greedy reordering or `protect_top_k`; paired significance tests and a cache-prefix continuity check remain necessary before deployment.

VIII. Artifact Availability and Team Contributions

The complete source repository is available at https://github.com/ubbx05/cache-aware-llm-router. The submission archive contains the router, all routing strategies, replay and calibration scripts, smoke tests, monitoring configuration, `SETUP.md`, dependency specification, and the LaTeX source and PDF of this paper; generated corpora, embeddings, traces, and live-run outputs are excluded. The raw TQuAD source is publicly available at https://github.com/TQuad/turkish-nlp-qa-dataset, and `bench/build_corpus.py` documents the deterministic text chunking and block-alignment transformation. Data fingerprints and trace manifests are supplied so an independently obtained copy can be checked against the experimental inputs.

Repository history reflects the following primary division of work. Umut Baran Boztaş developed the core proxy and replay pipeline, the per-worker/two-phase integration, and live deployment calibration. Saadet Cansu Baktıroğlu developed cache-measurement and adaptive, semantic, token-weighted, and quality-protection extensions and led the IEEE paper synthesis. Demir Doruk Dilek developed the replicated ordering and bookkeeping ablations, CUSUM calibration, schedule checks, and reproducibility documentation. All authors reviewed the integrated system, experimental interpretation, and final artifact.

IX. Conclusion and Future Work

This project implements and functionally validates a live, worker-specific two-step routing-and-ordering path on two replicas. Separately, at top-$k{=}10$, an order-only ablation under fixed routing finds that a global cache-history-informed greedy order raises the mean per-request cached-token fraction by 8.1 percentage points and lowers completion-path median TTFT by 27.9% relative to canonical order. Its primary contains-gold score is not separated from relevance order under the

prespecified spread rule. These results do not establish an incremental advantage from per-worker rather than global histories. The negative canonical and session-prefix results show that the benefit belongs to cache-informed ordering rather than reordering alone. Replicated tracker tests further show that completion-time bookkeeping degrades with concurrency while dispatch-time bookkeeping remains stable. Calibration and timing conventions therefore materially change routing conclusions.

Immediate work is therefore concrete. **(1)** Run the missing replicated system-level `per_worker_tree`-versus-global-greedy comparison at the calibrated 5,840-block capacity, with arm-specific router configuration, matched prompt serialisation, identical cold starts, and an end-to-end timer. Report cache reuse, TTFT p50/p95, answer quality, load CV, decision overhead, and failures; its joint design must be labelled separately from the pure-order arms. **(2)** Repeat at speedups 5 and 15 and on a second locality trace, aggregating cached token counts over each run. **(3)** Measure `protect_top_k` and extend the decision contract with query text and chunk token counts while making phase-one bookkeeping failure-aware. **(4)** Evaluate the DualMap-inspired arm and worker-specific ordering on an appropriate $n{>}2$ deployment. **(5)** Align adaptive modes on one overlap stream and either make calibrated CUSUM alarms an explicit control input or retain them strictly as telemetry. Raw runs, a complete environment manifest, paired quality tests, and prefix-continuity measurements must accompany those results.

REFERENCES

[1] TQuad, "Turkish NLP Question Answering Dataset," SQuAD-format Turkish question-answering data, accessed Aug. 15, 2026. https://github.com/TQuad/turkish-nlp-qa-dataset.

[2] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proc. 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023. https://arxiv.org/abs/2309.06180.

[3] K. Tan, R. Gu, and M. Li, "CacheWeaver: Cache-Aware Evidence Ordering for Efficient Grounded RAG Inference," *arXiv preprint arXiv:2606.19667*, 2026. https://arxiv.org/abs/2606.19667.

[4] Y. Yuan, P. Zuo, B. Wang, Z. Chen, Z. Tan, and Z. Yu, "DualMap: Enabling Both Cache Affinity and Load Balancing for Distributed LLM Serving," in *Proc. International Conference on Learning Representations (ICLR)*, 2026. https://arxiv.org/abs/2602.06502. Code: https://github.com/ASISys/DualMap.

[5] A. Dhasade, R. Guerraoui, A.-M. Kermarrec, D. Petrescu, R. Pires, M. Randl, and M. de Vos, "Efficient Federated Search for Retrieval-Augmented Generation Using Lightweight Routing," in *Proc. Distributed Applications and Interoperable Systems (DAIS)*, LNCS, vol. 16591, 2026, pp. 3–20. https://doi.org/10.1007/978-3-032-27358-1_1.

[6] V. Srivatsa, Z. He, R. Abhyankar, D. Li, and Y. Zhang, "Preble: Efficient Distributed Prompt Scheduling for LLM Serving," in *Proc. International Conference on Learning Representations (ICLR)*, 2025. https://arxiv.org/abs/2407.00023.

[7] R. Qin, Z. Li, W. He, M. Zhang, Y. Yang, W. Zheng, and Z. Zha, "Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving," in *Proc. USENIX Conference on File and Storage Technologies (FAST)*, 2025. https://arxiv.org/abs/2407.00079.

[8] NVIDIA, "Dynamo: A Datacenter Scale Distributed Inference Serving Framework," 2025. https://github.com/ai-dynamo/dynamo.

[9] C. Jin, Z. Zhang, X. Jiang, F. Liu, X. Liu, X. Liu, and X. Jin, "RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation," *ACM Transactions on Computer Systems*, vol. 44, no. 1, 2026. https://arxiv.org/abs/2404.12457.

[10] J. Yao, H. Li, Y. Liu, S. Ray, Y. Cheng, Q. Zhang, K. Du, S. Lu, and J. Jiang, "CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion," in *Proc. European Conference on Computer Systems (EuroSys)*, 2025. Best Paper Award. https://arxiv.org/abs/2405.16444.

[11] S. Lu, H. Wang, Y. Rong, Z. Chen, and Y. Tang, "TurboRAG: Accelerating Retrieval-Augmented Generation with Precomputed KV Caches for Chunked Text," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 6588–6601. https://arxiv.org/abs/2410.07590. Code: https://github.com/MooreThreads/TurboRAG.

[12] S. Agarwal, S. Sundaresan, S. Mitra, D. Mahapatra, A. Gupta, R. Sharma, N. J. Kapu, T. Yu, and S. Saini, "Cache-Craft: Managing Chunk-Caches for Efficient Retrieval-Augmented Generation," *Proc. ACM on Management of Data (SIGMOD)*, vol. 3, no. 3, 2025. https://arxiv.org/abs/2502.15734.

[13] A. Ricci Toniolo, A. Dinesh, and R. Thorstenson, "GORGO: Maximizing KV-Cache Reuse While Minimizing Network Latency in Cross-Region LLM Load Balancing," *arXiv preprint arXiv:2602.11688*, 2026. https://arxiv.org/abs/2602.11688.

[14] F. Bang, "GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings," in *Proc. 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS)*, 2023. https://aclanthology.org/2023.nlposs-1.24/.

[15] J. Li, C. Xu, F. Wang, I. M. von Riedemann, C. Zhang, and J. Liu, "SCALM: Towards Semantic Caching for Automated Chat Services with Large Language Models," *arXiv preprint arXiv:2406.00025*, 2024. https://arxiv.org/abs/2406.00025.

[16] W. Gill, M. Elidrisi, P. Kalapatapu, A. Ahmed, A. Anwar, and M. A. Gulzar, "MeanCache: User-Centric Semantic Caching for LLM Web Services," *arXiv preprint arXiv:2403.02694*, 2024. https://arxiv.org/abs/2403.02694.